Appendix R7A: Dimensionality - Regularization

Penalized Models: Ridge and LASSO Regressions

J. Alberto Espinosa

2025-01-04

This Quarto document contains R code examples about predictive modeling methods based on regularization, also known as shrinkage or penalized models. I will describe three methods – Ridge Regression and LASSO, and Elastic Nets, which is a blend of Ridge and LASSO. These models aim to correct issues of dimensionality, such as severe multicollinearity. An important thing to note is that these methods shrink the regression coefficients intentionally to bias the predictor coefficients to improve the model variance. Naturally, the more we shrink the coefficients, the more we bias the model. Cross-validation will help us identify the optimal amount of shrinkage in when the model variance is minimized. It is important to note that the more we shrink the model coefficients, the less useful the model becomes for interpretation and explainability. For the corresponding conceptual material associated with these appendinces, consult the main book Predictive Analytics and Machine Learning for Managers. Chat with the PAML4M book and its related lecture slides and R scripts with its companion PAML4M ChatBot.

Table of contents

Technical Note: All procedures involving machine learning and cross-validation are based on random sampling. Therefore, results may change when you run the analysis multiple times, depending on the kind of random number generator `RNGkind()` and the seed selected. The results in this script may not match the outputs in the main text of the book. Follow the chapter and this script appendix independently. Let's first set the random number generator default

```
RNGkind(sample.kind = "default") # To use the R default RNG
```

Dimensionality

The issue of dimensionality is often referred to as **curse of dimensionality**. As you add more predictors and complexity to a model specification, it will generally fit the data better and most fit statistics will improve. In some cases, the fit may even be perfect. For example, if you grow a tree until it has the same number of branches as you have data points, the fit will be perfect because the model will touch every single data point. Similarly, if you fit a smoothing spline model, you can set your parameters so that the resulting regression model with a jagged line that touches every single point in the data set. However, such models are **over-fitting** and tend to not perform well with new data due to extreme variance, making fit statistics like R-squared, MSE and 2LL to improve **artificially**, giving the false impression that the model is good. However, if we CV test the model with different data, over-fitted models will have higher variance, and these predictions will not be as accurate.

Over-fitting and multicollinearity are two of the many problems of dimensionality. The methods presented in this section are ideally suited to address dimensionality issues. There are many types of methods that address and correct for dimensionality. The 2 types of methods we will cover in this class are: (1) Regularization or Shrinkage (covered in this script); and (2) Dimension Reduction (covered in the Appendix R7B).

Regularization (i.e., Shrinkage or Penalized) Methods.

Regularized, shrinkage and penalized models are the same thing and they involve artifically shrinking regression coefficients to reduce the effect of multicollinearity on the model's variance. Generally speaking, Ridge and LASSO work best when the number of predictors is quite large (even thousands) and the data suffers from severe multicollinearity.

When coefficients are shrunk, the model's **bias** increases. On the upside, shrinkage reduces the model variance. The amount of shrinkage is controlled by a **shrinkage factor** lambda, which is a **tuning parameter**, meaning that you can shrink coefficients to various degrees and then select the shrinkage that yields the best cross-validation testing results (i.e., best predictive accuracy and optimal model variance). In this section I describe two popular regularization approaches: **Ridge** and **LASSO** (Least Absolute Shrinkage and Selection Operator) regressions. They both aim to shrink the coefficients, such that the coefficients of the less important variables become very small and, therefore, have little influence on predictions. The magnitude of the shrinkage can be controlled with the shinkage factor "Lambda", such that when `Lambda = 0`, both Ridge and LASSO results are identical to OLS or GLM model, that is, a regression model with no shrinkage. In contrast, when `Lambda = infinite`, both methods also yield the same results as the **Null** model because all regression coefficients are squashed to 0. The difference between Ridge and LASSO is that as Lambda grows, Ridge coefficients become smaller, but not 0, except when Lambda is infinite. In contrast, LASSO shrinks coefficients to 0 along the way, dropping their respective predictors from the model as Lambda grows.

Because Lambda causes the coefficients to shrink, the larger the Lambda, the more **biased** the regression coefficients are compared to OLS or GLM coefficients. Generally, as Lambda grows, Ridge and LASSO improve the model's predictive accuracy up to a point, after which the predictive accuracy declines. An optimal model is one with a Lambda value that minimizes the model's CV test deviance. However, if the analytics goal is interpretability, it would OK to select a model with a smaller Lambda that has an acceptable CV test deviance.

How do you know if your model is biased? One way to quickly inspect for bias is to put the regression coefficients for various shrinkage factors in columns, from no shrinkage to extremely large shrinkage, and observe how the coefficients change from column to column. If a coefficient is relatively constant and stable, then it has little bias. If a coefficient fluctuates widely and even changes signs, then it has a lot of bias. Overall, the closer the coefficients are in value to the OLS coefficients, the less biased the model is and, consequently the more interpretable.

IMPORTANT: with shrinkage methods like Ridge regression, p-values no longer relevant, because all coefficients are retained in the model, although shrunk (i.e., biased). The actual magnitude of the coefficients tells you how important the corresponding variable is in the prediction. Similarly, the R-squared values are not reported, but for quantitative outcome models, you can use the deviance ratio illustrated above as an R squared.

Technical Note: One way to get coefficients in Ridge or LASSO regression representing variable importance is to standardize all variables. Ridge and LASSO do this automatically, but then standardization the coefficients for the output. But if you standardize the variables yourself, you will get standardized coefficients, which are unaffected

by variable scales. Consequently, predictors with larger standardized coefficients will have more weight on the predictions, thus providing a measure of variable importance.

Ridge Regression

Let's run a Ridge regression with several Lambdas and find the best Lambda that minimizes the 10FCV MSE. We will be using the **glmnet()** function from the **{glmnet}** R package. Let's start by loading the necessary packages.

```
library(glmnet)
library(ISLR) # Contains the Hitters baseball player salary data set
# This data set has several missing values, let's omit them
Hitters <- na.omit(Hitters)
options(scipen = 4) # Minimize use of scientific notation
```

Technical Note: the **glmnet()** function has a different syntax than other functions like `lm()` and `glm()`. It requires defining an **X matrix** (of the predictors) and a **Y** vector (of the response values). That is, rather than using the familiar `y ~ x1 + x2 etc.` formula syntax, we will use the `(X, Y, etc)` syntax, where X is the model matrix of predictors and Y is the vector of outcomes. We will use the `model.matrix()` function in the `{stats}` package to create the model's predictor matrix. The `model.matrix()` function will also convert all categorical variables to their respective dummy variables.

** Technical Note:** Notice below that I added `[,-1]` at the end of the `model.matrix()` function. This is necessary because the `model.matrix()` function will include a column full of 1's as the first column, which represents the intercept. But Ridge also renders an intercept. So, if you don't remove this first column of 1's, the Ridge model will have 2 intercepts and the coefficients will be slightly off.

Let's start by creating the **x** predictor matrix and the **y** outcome vector:

```
x <- model.matrix(Salary ~ ., data = Hitters)[ ,-1]
x[1:10, 1:10] # Take a look at the first 10 rows and columns
```

```
                  AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
-Alan Ashby         315   81     7   24  38    39    14   3449   835     69
-Alvin Davis        479  130    18   66  72    76     3   1624   457     63
-Andre Dawson       496  141    20   65  78    37    11   5628  1575    225
-Andres Galarraga   321   87    10   39  42    30     2    396   101     12
-Alfredo Griffin    594  169     4   74  51    35    11   4408  1133     19
-Al Newman          185   37     1   23   8    21     2    214    42      1
-Argenis Salazar    298   73     0   24  24     7     3    509   108      0
-Andres Thomas      323   81     6   26  32     8     2    341    86      6
-Andre Thornton     401   92    17   49  66    65    13   5206  1332    253
-Alan Trammell      574  159    21  107  75    59    10   4631  1300     90
```

```
y <- Hitters$Salary # y vector with just the outcome variable
y[1:10] # Take a look at the first 10 values
```

```
 [1]  475.000  480.000  500.000   91.500  750.000   70.000  100.000   75.000
 [9] 1100.000  517.143
```

The `glmnet()` function is used to fit, Ridge, LASSO and Elastic Net models. It uses the parameter `alpha =` to indicate which model to fit, with a value of **0** for a **Ridge Regression**, **1** for **LASSO** and any value between **0-1**

for Elastic Net. `alpha` is the proportional weight given to **LASSO** and (`1 - alpha`) is the proportional weight given to **Ridge**.

Technical Note: By defalut, `glmnet()` estimates models with 100 different Lambda values, ranging from very large to very small (in descending order). You can estimate less or more Lambda values with the `nlambda =` parameter)

```
options(scipen = 999) # Disable scientific notation
ridge.mod <- glmnet(x, y,
                    alpha = 0)
```

Note About GLM Models: To fit a Ridge regression with a binomial outcome (e.g., **Logistic**), use the parameter `family = "binomial"`. For **count** outcomes, use the parameter `family = "poisson"`. The interpretation of the resulting coefficients is the same as with Logistic and Count data models.

Let's take a quick look at the first 10 Lambdas and their **Deviance Ratios** (i.e., proportion of Null deviance explained, like an R Squared), rounded to decimals). The Lambdas are sorted from largest (close to the Null model) to smallest (close to OLS). Naturally, the explained variance of the model (relative to the Null model) increases as lambda decreases because we are getting closer to the OLS model. But this will change once we CV test the model.

```
round(cbind("Lambda" = ridge.mod$lambda,
            "Deviance Ratio" = ridge.mod$dev.ratio)[1:10, ],
      digits = 4)
```

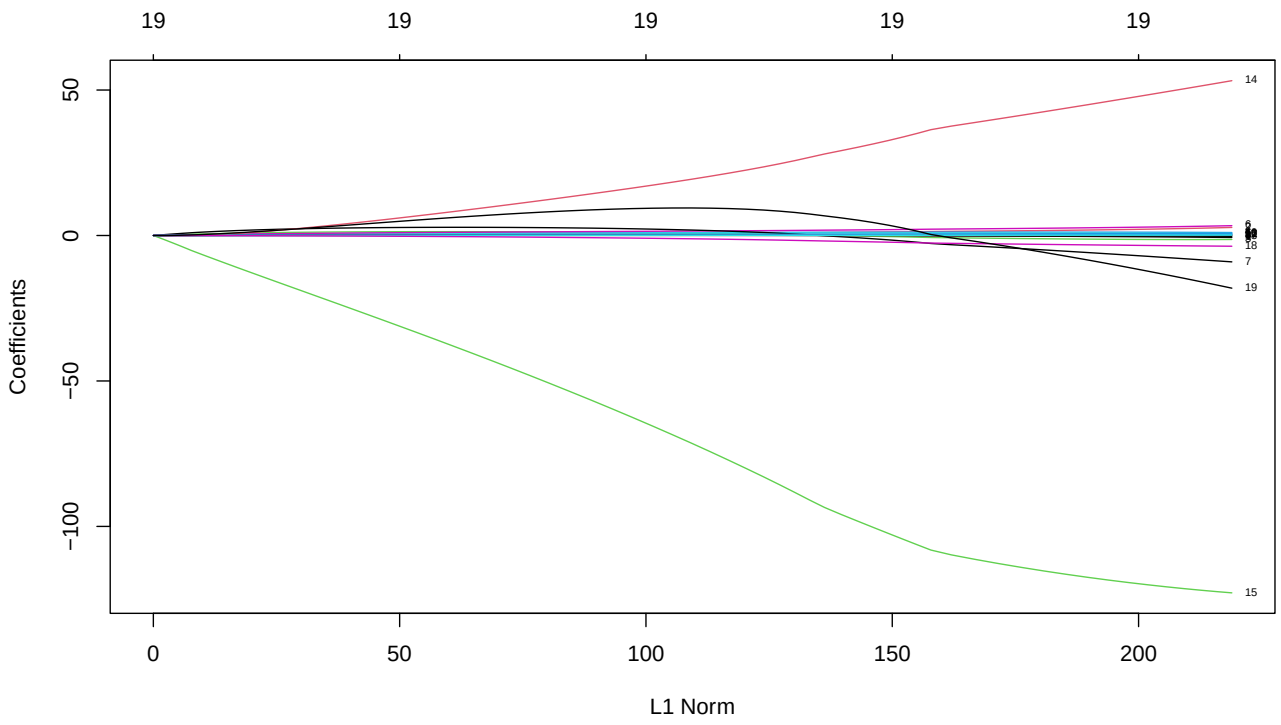
```
       Lambda Deviance Ratio
 [1,] 255282.1         0.0000
 [2,] 232603.5         0.0116
 [3,] 211939.7         0.0127
 [4,] 193111.5         0.0139
 [5,] 175956.0         0.0153
 [6,] 160324.6         0.0167
 [7,] 146081.8         0.0183
 [8,] 133104.3         0.0200
 [9,] 121279.7         0.0219
[10,] 110505.5         0.0239
```

Note: Ridge regression standardizes the coefficients because the scale of the variables can affect the proportion of shrinkage across coefficients. However, Ridge regression can be run without standardizing coefficients simply by specifying the parameter `standardize = FALSE`. The default is TRUE.

L2 Norm

L2 Norm is a measure of the overall shrinkage of the model. It is the square root of the sum of the squared coefficients in a Ridge regression. As Lambda increases, all coefficients shrink, so the L2 Norm decreases too. `L2 Norm = square root of the sum of the model's squared coefficients`. L2 Norm is mostly used as a preliminary visual analysis of the shrinkage patter not the model. For example, notice in the graph below how the coefficients shrink. Some get pretty small fairly quickly, but no coefficient becomes 0 until Lambda is infinitely large. Also notice how some coefficients change in size more than others and some even change signs, providing some evidence of bias. Also, notice the repeated value of 19 at the top of the graph. This means that the model has 19 predictors at every shrinkage level. As we will see shortly, this will not be the case with LASSO.

```
# Note: the x axis is Labeled L1 Norm, but the prefix L1 is generally used for LASSO.
plot(ridge.mod, label = T)
```



To compute the **L2 Norm** for any lambda, for example 50 (notice we remove the first coefficient, which is the intercept):

```
l2.norm.50 <- sqrt(sum(coef(ridge.mod)[-1, 50] ^ 2))
l2.norm.50
```

```
[1] 22.53415
```

Optimal Shrinkage with CV Testing

Let's find the best Lambda shrinkage value that minimizes the CV test MSE using the `cv.glmnet()` in the `{glmnet}` R package. The syntax in the `cv.glmnet()` function is the same as for the `glmnet()` function, except that it has an additional parameters, `nfolds =` to change the CV test default from **10FCV** to other number of folds.

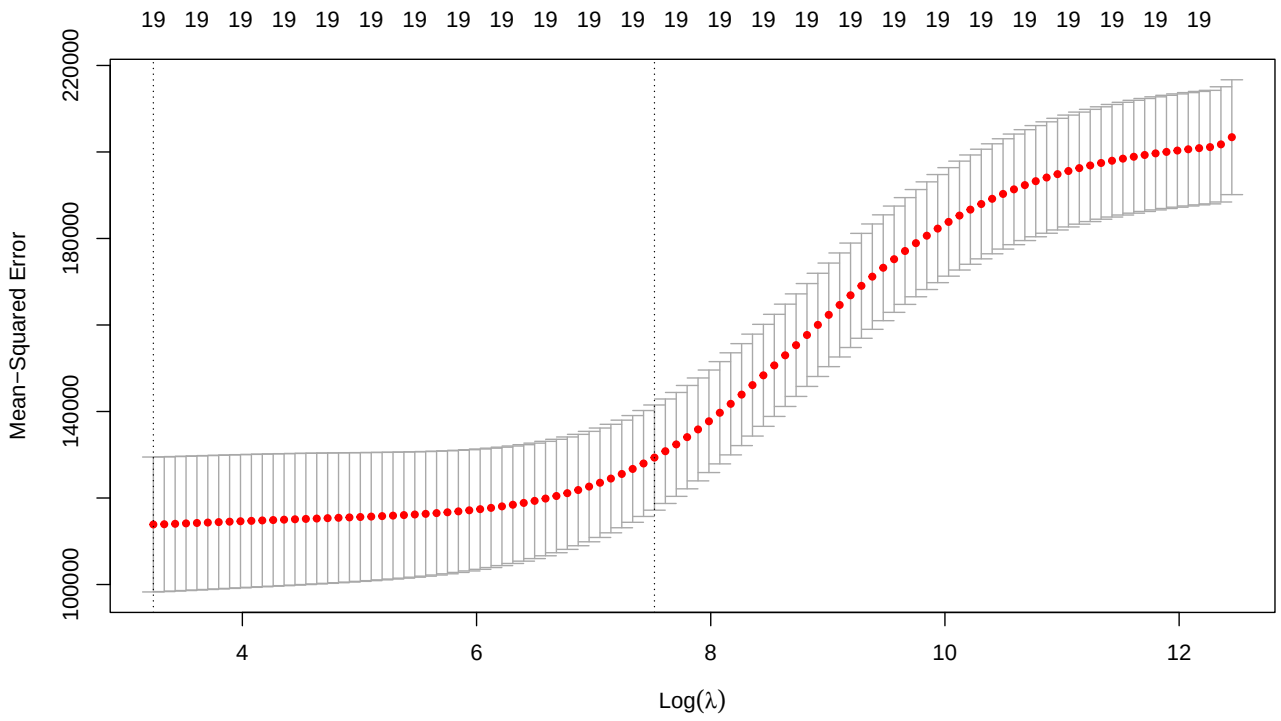
```
set.seed(1) # Arbitrary,to get repeatable results
cv.10Fold <- cv.glmnet(x, y,
                       alpha = 0) # 10FCV is the default

# First 10 Lambdas (remove the index to print all lambdas)
cbind("Lambda" = cv.10Fold$lambda,
      "10FCV MSE" = cv.10Fold$cvm)[1:10,]
```

```
      Lambda 10FCV MSE
[1,] 255282.1  203414.3
[2,] 232603.5  201753.4
[3,] 211939.7  201122.4
[4,] 193111.5  200881.5
```

```
 [5,] 175956.0  200618.5
 [6,] 160324.6  200331.2
 [7,] 146081.8  200017.6
 [8,] 133104.3  199675.6
 [9,] 121279.7  199302.8
[10,] 110505.5  198896.5
```

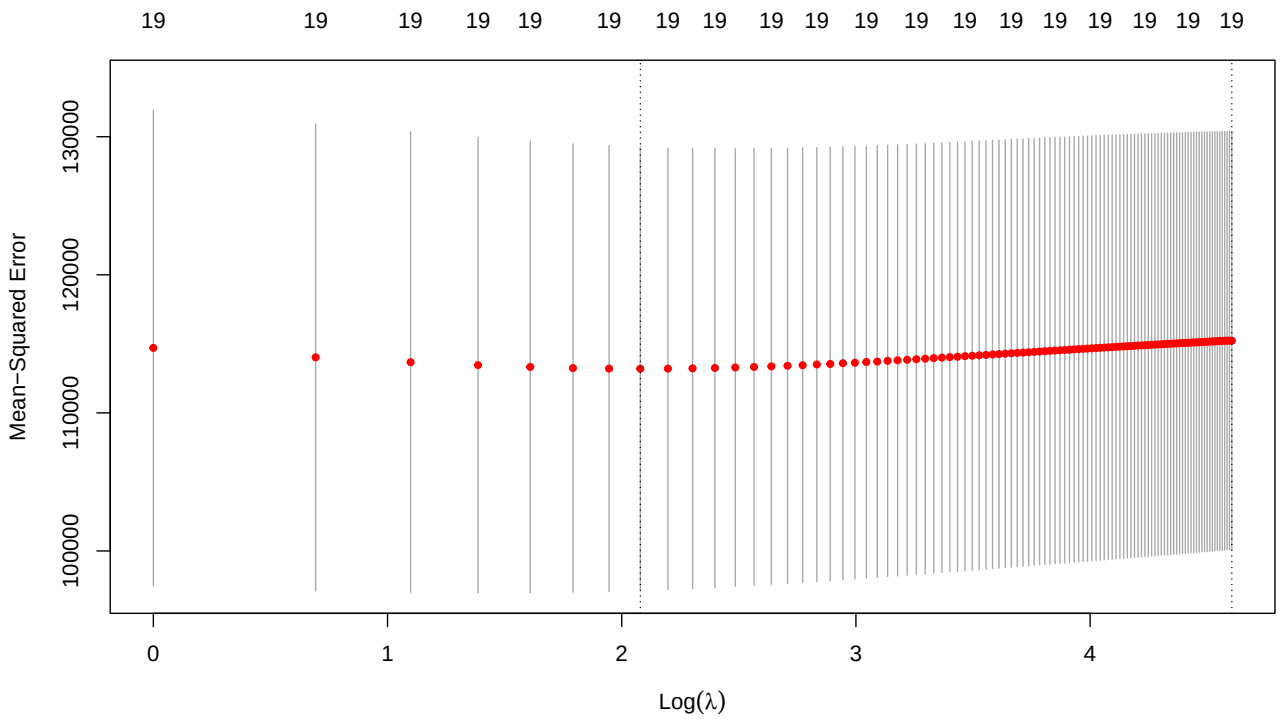
Let's plot all the Lambdas against their 10FCV MSE

```
plot(cv.10Fold)
```



The plot suggests that the MSE declines as the shrinkage lambda decreases. There is no clearly visible minimum in the graph. But we can evaluate this more carefully, by exploring several small lambdas, say from 1 to 100.

```
set.seed(1)
cv.10Fold <- cv.glmnet(x, y,
                       alpha = 0,
                       lambda = seq(0, 100)) # 10FCV is the default
plot(cv.10Fold)
```



The new plot shows some possible minimum value for the MSE around `log(lambda) = 2`. Let's find it more precisely, quantitatively. The best best Lambda is the one that minimizes the 10FCV test MSE. The `cv.glmnet()` output object has the attribute `lambda.min` that (conveniently) stores the Lambda with the smallest CV test MSE. Let's display it, along with its log to match the plot display. Let's also compute the corresponding CV test MSE by taking the CV vector '$cvm`(has all MSE's for all Lambdas) and finding its minimum value. I then display the results. And, sure enough, the best lambda is 8 with`log(lambda) = 2.079`and`10FCV = 113,198.6'.

```
best.lambda <- cv.10Fold$lambda.min
min.mse <- min(cv.10Fold$cvm)

cbind("Best Lambda" = best.lambda,
      "Log(Lambda)" = log(best.lambda),
      "Best 10FCV MSE" = min.mse)
```

```
     Best Lambda Log(Lambda) Best 10FCV MSE
[1,]           8    2.079442       113198.6
```

Extracting Ridge Coefficients

You can extract Ridge regression coefficients for any level of Lambda shrinkage with the `coef()` function and the `s=` parameter.

Technical note: when you use `s = 0` (or any other fixed value), the `coef()` or `predict()` functions will not use the exact lambda value of 0, but one of the 100 lambdas used when fitting the model or will do some interpolation. So the results for OLS using `s = 0` will be close enough but not exactly the same as the ones for OLS. In order to get **exact** coefficients, you need to use the parameters `x = x, y = y, exact = T, s = 0`. In essence, these parameters tell the `coef()` function to estimate the ridge model for the exact lambda value `s = 0` , which will extract exact coefficients rather than interpolated one.

Similar to OLS with **`Lambda s = 0`**:

```
options(scipen = 4)

ridge.0 <- round(coef(ridge.mod,
                      x = x, y = y,
                      exact = T,
                      s = 0),
                 digits = 4)
```

For the optimal Lambda **`s = best.lambda`**., also with `exact = T`:

```
ridge.best <- round(coef(ridge.mod,
                         x = x, y = y,
                         exact = T,
                         s = best.lambda),
                    digits = 4)
```

For some arbitrary **`Lambda s = 50`** (without exact coefficients):

```
ridge.50 <- round(coef(ridge.mod,
                       s = 50),
                  digits = 4)
```

For some extreme shrinkage value, close to the **Null** model (without exact coefficients):

```
ridge.huge <- round(coef(ridge.mod,
                         s = 10 ^ 20),
                    digits = 4)
```

Let's display all coefficients

```
ridge.all <- cbind(ridge.0,
                   ridge.best,
                   ridge.50,
                   ridge.huge)
colnames(ridge.all) <- c("OLS, Lambda = 0",
                         "Best Lambda = 8",
                         "Lambda = 50",
                         "Almost Null")
ridge.all
```

```
20 x 4 sparse Matrix of class "dgCMatrix"
            OLS, Lambda = 0 Best Lambda = 8 Lambda = 50 Almost Null
(Intercept)        161.9687        130.9604     48.2165    535.9259
AtBat               -1.9483         -1.3235     -0.3539      0.0000
Hits                 7.3496          4.6658      1.9532      0.0000
HmRun                4.1752         -0.2302     -1.2851      0.0000
Runs                -2.2542          0.2324      1.1563      0.0000
RBI                 -1.0060          0.3171      0.8088      0.0000
Walks                6.1738          4.6457      2.7098      0.0000
Years               -2.8956        -10.7666     -6.2029      0.0000
CAtBat              -0.1878         -0.0330      0.0061      0.0000
CHits                0.2157          0.1779      0.1071      0.0000
CHmRun              -0.0528          0.7155      0.6291      0.0000
```

```
CRuns              1.4041          0.5443      0.2173      0.0000
CRBI               0.7622          0.3393      0.2153      0.0000
CWalks            -0.7912         -0.5067     -0.1489      0.0000
LeagueN           63.5164         59.9218     45.8626      0.0000
DivisionW       -116.7604       -123.7976   -118.2304      0.0000
PutOuts            0.2814          0.2761      0.2502      0.0000
Assists            0.3767          0.2534      0.1208      0.0000
Errors            -3.4180         -3.8362     -3.2771      0.0000
NewLeagueN       -25.6543        -26.3324     -9.4235      0.0000
```

As you can see from the output above, the OLS coefficients are the same as those for the Ridge model with `Lambda = 8` (minimal shrinkage). The coefficients appear to be identical, but that is due to rounding. With sufficient decimals you should see some very small differences. You can also see that an arbitrary value of `Lambda = 50` biases the coefficients a little, but not by much. In contrast, the large Lambda yields an almost Null model. I say "almost" because Ridge does not drop the coefficients, and with sufficient decimals you would see very small non-zero coefficients. But for practical reasons, a Lambda = 10 ^ 20 yields very similar results to the Null model.

You can also get coefficients for a specific group of lambdas, any sequence of lambdas or a particular lambda number:

```
ridge.mod.group <- glmnet(x, y,
                          alpha = 0,
                          lambda = c(10 ^ 20, 10000, 10, 0))
round(coef(ridge.mod.group),
      digits = 4) # Take a look
```

```
20 x 4 sparse Matrix of class "dgCMatrix"
                    s0       s1        s2        s3
(Intercept) 535.9259 392.7312  122.3737  161.9747
AtBat         0.0000   0.0411   -1.2068   -1.9484
Hits          0.0000   0.1545    4.2932    7.3497
HmRun         0.0000   0.5814   -0.4841    4.1744
Runs          0.0000   0.2574    0.4559   -2.2543
RBI           0.0000   0.2670    0.3972   -1.0056
Walks         0.0000   0.3236    4.4015    6.1740
Years         0.0000   1.2262  -10.7809   -2.8965
CAtBat        0.0000   0.0035   -0.0226   -0.1878
CHits         0.0000   0.0130    0.1652    0.2158
CHmRun        0.0000   0.0974    0.6898   -0.0523
CRuns         0.0000   0.0260    0.4727    1.4041
CRBI          0.0000   0.0269    0.3338    0.7620
CWalks        0.0000   0.0278   -0.4597   -0.7913
LeagueN       0.0000   0.1688   58.9833   63.5176
DivisionW     0.0000  -7.0644 -124.1991 -116.7595
PutOuts       0.0000   0.0186    0.2742    0.2814
Assists       0.0000   0.0029    0.2369    0.3767
Errors        0.0000  -0.0245   -3.8505   -3.4180
NewLeagueN    0.0000   0.3930  -25.4166  -25.6552
```

Or display coefficients for a sequence of Lambdas

```
ridge.mod.sequence <- glmnet(x, y,
                             alpha = 0,
                             lambda = seq(1, 5))
round(coef(ridge.mod.sequence), digits = 4) # Take a look
```

```
20 x 5 sparse Matrix of class "dgCMatrix"
                    s0        s1        s2        s3        s4
(Intercept)  143.9106  150.3870  154.8083  159.1816  162.4634
AtBat         -1.5387   -1.6311   -1.7190   -1.8126   -1.9006
Hits           5.4047    5.6976    6.0451    6.4387    6.8769
HmRun          0.5463    0.7278    1.1689    1.7513    2.6002
Runs          -0.1905   -0.3778   -0.6670   -1.0199   -1.5030
RBI            0.0863    0.0408   -0.0849   -0.2531   -0.5051
Walks          5.0681    5.2665    5.4616    5.6794    5.9210
Years        -10.3670  -10.3179   -9.5292   -8.4591   -6.5952
CAtBat        -0.0495   -0.0580   -0.0739   -0.0942   -0.1270
CHits          0.1815    0.2031    0.2098    0.2159    0.2236
CHmRun         0.6451    0.7134    0.6671    0.5850    0.4177
CRuns          0.6622    0.7101    0.8042    0.9205    1.0939
CRBI           0.3905    0.3705    0.4008    0.4474    0.5344
CWalks        -0.5767   -0.6132   -0.6492   -0.6891   -0.7358
LeagueN       60.8330   61.4366   61.8941   62.3310   62.8899
DivisionW   -123.1024 -122.6512 -121.8053 -120.7689 -119.2272
PutOuts        0.2783    0.2796    0.2806    0.2815    0.2820
Assists        0.2799    0.2926    0.3072    0.3240    0.3458
Errors        -3.7777   -3.7555   -3.7034   -3.6377   -3.5495
NewLeagueN   -27.3382  -27.9432  -27.9504  -27.7946  -27.2580
```

And also display coefficients for a particular sequence order of Lambda

```
ridge.mod.sequence$lambda[3] # e.g., display the third Lambda
```

```
[1] 3
```

```
round(coef(ridge.mod.sequence)[ ,3],
      digits = 4) # Its coefficients
```

```
(Intercept)       AtBat        Hits       HmRun        Runs         RBI
   154.8083     -1.7190      6.0451      1.1689     -0.6670     -0.0849
      Walks       Years      CAtBat       CHits      CHmRun       CRuns
     5.4616     -9.5292     -0.0739      0.2098      0.6671      0.8042
       CRBI      CWalks     LeagueN   DivisionW     PutOuts     Assists
     0.4008     -0.6492     61.8941   -121.8053      0.2806      0.3072
     Errors  NewLeagueN
    -3.7034    -27.9504
```

Predicting with Ridge

We can use the **predict()** function to obtain Ridge regression coefficients using the attribute `type = "coefficients"`. Let's find the Ridge regression coefficients for the best lambda:

```
predict(ridge.mod,
        s = best.lambda,
        type = "coefficients")
```

```
20 x 1 sparse Matrix of class "dgCMatrix"
                       s1
```

```
(Intercept)   81.126931905
AtBat         -0.681595889
Hits           2.772311585
HmRun         -1.365680115
Runs           1.014826482
RBI            0.713022449
Walks          3.378557597
Years         -9.066800407
CAtBat        -0.001199478
CHits          0.136102881
CHmRun         0.697995816
CRuns          0.295889602
CRBI           0.257071062
CWalks        -0.278966595
LeagueN       53.212720343
DivisionW   -122.834451506
PutOuts        0.263887567
Assists        0.169879575
Errors        -3.685645337
NewLeagueN   -18.105095956
```

Now let's use the `predict()` function to make actual predictions. To do predictions with Ridge regression, you need to store the observations you want to do predictions for in a **new X** and specify it with the `newx =` parameter. This `newx` can either be new data for which you want to make predictions, or some test sub-sample to evaluate the model's accuracy. For the purposes of this example, let's just draw a random test sub-sample with 5% of the existing observations, using the best lambda we found above from the same **x** predictor matrix above.

```
set.seed(1)
test <- sample(1:nrow(x),
               0.05 * nrow(x))
x.test <- x[test, ] # Extract test sub-sample
ridge.pred <- predict(ridge.mod, # predict outcomes
                      s = best.lambda,
                      newx = x.test)
ridge.pred # Take a look
```

```
                       s1
-Mike Heath      463.6768
-John Russell    385.6552
-Pete Incaviglia 307.2966
-Glenn Davis     776.5389
-Frank White     859.7506
-Rafael Santana  347.9036
-Chili Davis     680.8958
-Herm Winningham 213.7646
-Robby Thompson  390.6993
-Joe Carter      647.9247
-Spike Owen      344.9689
-Mike Easler     673.2900
-Chris Bando     320.4533
```

Ridge and Logistic Regression

You can fit a Ridge shrinkage model on a Logistic Regression simply by adding the parameter `family = "binomial"` to both functions, `glmnet()` and `cv.glmnet()`. Selecting the best lambda is identical to quantitative Ridge models.

The coefficient interpretation in Ridge Logistic is the same as for Logistic Regression interpretation based on log-odds effects.

To illustrate Ridge Logistic, we can access the `SAHeart.data` dataset in Prof. Robert Tibshirani's web site, which contains variables associated with coronary heart disease. The dataset and variable descriptions are available at: https://rdrr.io/cran/ElemStatLearn/man/SAheart.html.

Once you have downloaded the data set , you can open it directly. I downloaded the file in CSV format with the name **Heart.csv**, which can be read as follows:

```
heart <- read.table("Heart.csv", sep = ",", head = T)
```

Once you have the data in the heart data frame object, the next step is to create the predictor matrix **x** and the outcome vector **y**. The outcome variable is binary, **chd = 1** for disease, **0** otherwise)

```
x <- model.matrix(chd ~ ., data = heart)[,-1]
y <- heart$chd
```

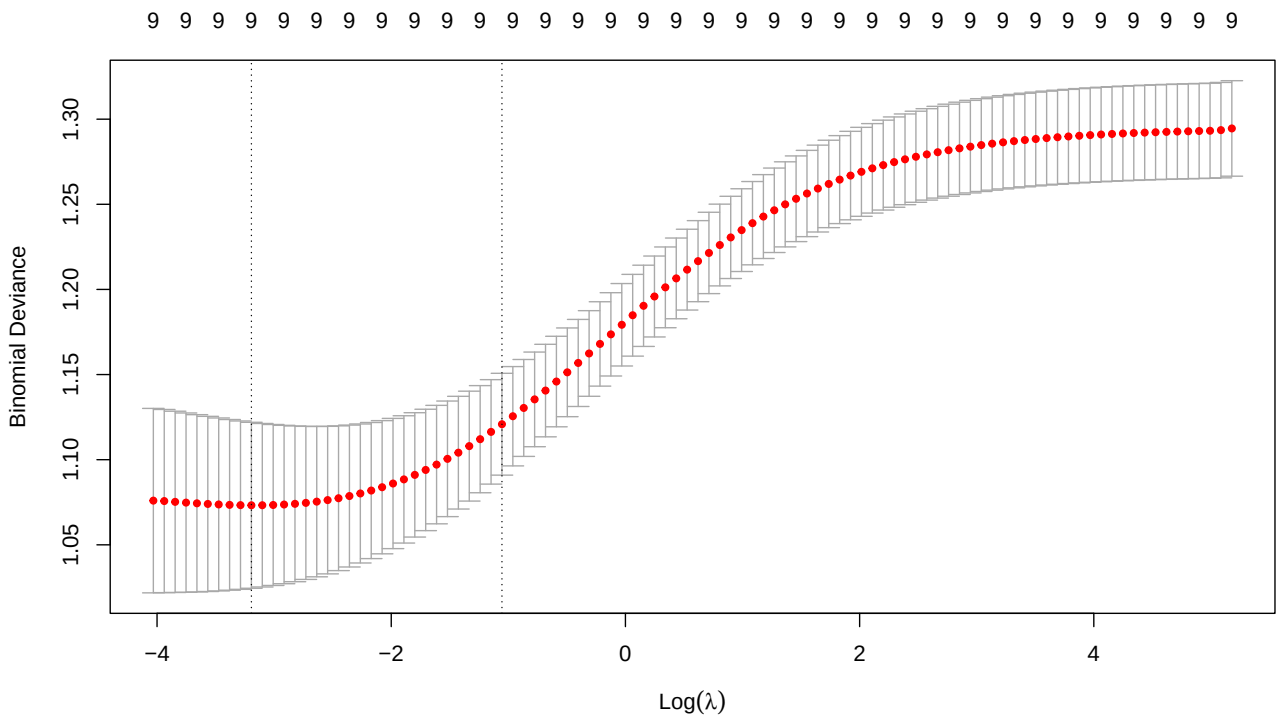
We then follow the same steps as for quantitative Ridge regressions, described above, with three differences:

- We use the `family = "binomial"` parameter
- The CV test reported in `$cvm` is deviance using the `2LL = -2 * Log-Likelihood` fit statistic for GLM models
- If we want to use **classification error** as the CV test deviance, we simply include the `measure.type = "class"` parameter.

```
# Fit the Ridge Logistic model
ridge.logit=glmnet(x, y,
                   alpha = 0,
                   family = "binomial")

# Find best lambda:
set.seed(1)
cv.10Fold.logit <- cv.glmnet(x, y,
                             alpha = 0,
                             family = "binomial")

# Report (first 10 Lambdas to minimize the printout) and plot
cbind("Lambda" = cv.10Fold.logit$lambda,
      "10-Fold CV Deviance" = cv.10Fold.logit$cvm)[1:10,]
```

```
        Lambda 10-Fold CV Deviance
 [1,] 177.45951            1.294554
 [2,] 161.69449            1.293559
 [3,] 147.33000            1.293179
 [4,] 134.24161            1.293039
 [5,] 122.31596            1.292886
 [6,] 111.44974            1.292717
 [7,] 101.54886            1.292533
 [8,]  92.52753            1.292331
 [9,]  84.30764            1.292109
[10,]  76.81798            1.291866
```

```
plot(cv.10Fold.logit)
```



Now let's find the **Best Lambda**.

```
best.lambda.logit <- cv.10Fold.logit$lambda.min
log(best.lambda) # Log to spot it in the plot
```

```
[1] 2.079442
```

```
min.mse.logit <- min(cv.10Fold.logit$cvm)
```

And then display the results

```
cbind("Best Lambda" = best.lambda.logit,
      "Log(Lambda)" = log(best.lambda.logit),
      "Best 10FCV Deviance" = min.mse.logit)
```

```
     Best Lambda Log(Lambda) Best 10FCV Deviance
[1,]  0.04099545   -3.194294            1.073231
```

Then let's display coefficients

```
# Logit Model
ridge.logit.coef.0 <- coef(ridge.logit,
                           s = 0)
```

Alternatively, you can use the following to extract coefficients:

```
predict(ridge.logit, s = best.lambda.logit, type = "coefficients")
```

```
10 x 1 sparse Matrix of class "dgCMatrix"
                            s1
(Intercept)    -5.3186664789
sbp             0.0062893712
tobacco         0.0704757286
ldl             0.1421137485
adiposity       0.0167321735
famhistPresent  0.7664077959
typea           0.0280980909
obesity        -0.0376375020
alcohol         0.0003693287
age             0.0339089740
```

Best Lambda Model

```
ridge.logit.coef.best <- coef(ridge.logit,
                              s = best.lambda.logit)
```

Almost Null Model

```
ridge.logit.coef.large <- coef(ridge.logit,
                               s = 10 ^ 20)
```

Display all coefficients

```
all.coefs <- round(cbind(ridge.logit.coef.0,
                         ridge.logit.coef.best,
                         ridge.logit.coef.large),
                   digits = 4)
colnames(all.coefs) <- c("Logistic",
                         "Best Lambda",
                        "Almost Null Model")
all.coefs
```

```
10 x 3 sparse Matrix of class "dgCMatrix"
               Logistic Best Lambda Almost Null Model
(Intercept)     -5.7222     -5.3187           -0.6353
sbp              0.0065      0.0063            0.0000
tobacco          0.0754      0.0705            0.0000
ldl              0.1575      0.1421            0.0000
adiposity        0.0174      0.0167            0.0000
famhistPresent   0.8462      0.7664            0.0000
typea            0.0334      0.0281            0.0000
obesity         -0.0491     -0.0376            0.0000
alcohol          0.0002      0.0004            0.0000
age              0.0391      0.0339            0.0000
```

LASSO Regression

To fit LASSO regression models we follow the exact same steps as above, except that we use the parameter `alpha = 1` rather than 0.

We can also fit Elastic Net regression models using `0 < alpha < 1`, which is a hybrid between Ridge and LASSO.

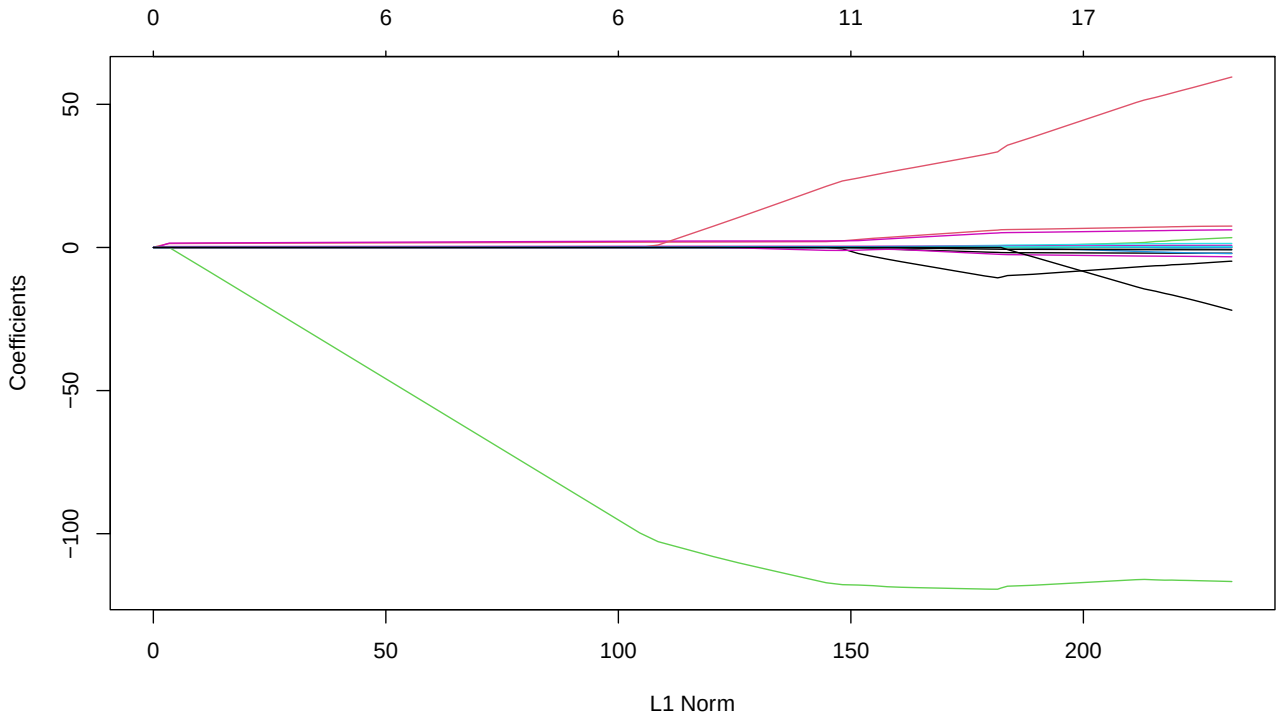
Again, the one difference between Ridge regression and LASSO is that coefficients can shrink significantly with Ridge regression, but they never become 0 (except when lambda is infinite). In contrasts, some LASSO coefficients can shrink to 0. The more you shrink, the more coefficients drop out of the model. Let's re-create the x predictor matrix and y outcome vector for the Hitters data set.

We already did this above, but let's repeat it here for convenience.

```
library(glmnet)
library(ISLR) # Contains the Hitters baseball player salary data set
Hitters <- na.omit(Hitters)
options(scipen = 4)

x <- model.matrix(Salary ~ .,data = Hitters)[ ,-1]
y <- Hitters$Salary
```

Now let's fit the LASSO model and graph the **L1 Norm**. L1 Norm is the sum of the absolute values of the coefficients in a LASSO regression. Like L2 Norm, it measures the total amount of shrinkage obtained by a particular Lambda value used, but using absolute values, rather than squared values. The larger the L1 the smaller the shrinkage.

```
lasso.mod <- glmnet(x, y,
                    alpha = 1)
plot(lasso.mod) # Plot the L1 Norm against the coefficients
```



Optimal LASSO Lambda

```
set.seed(1) # To get repeatable results
cv.10Fold.lasso <- cv.glmnet(x, y,
                             alpha = 1) # 10-Fold is the default
# Use the parameter nfolds=xx to change the number of folds
```

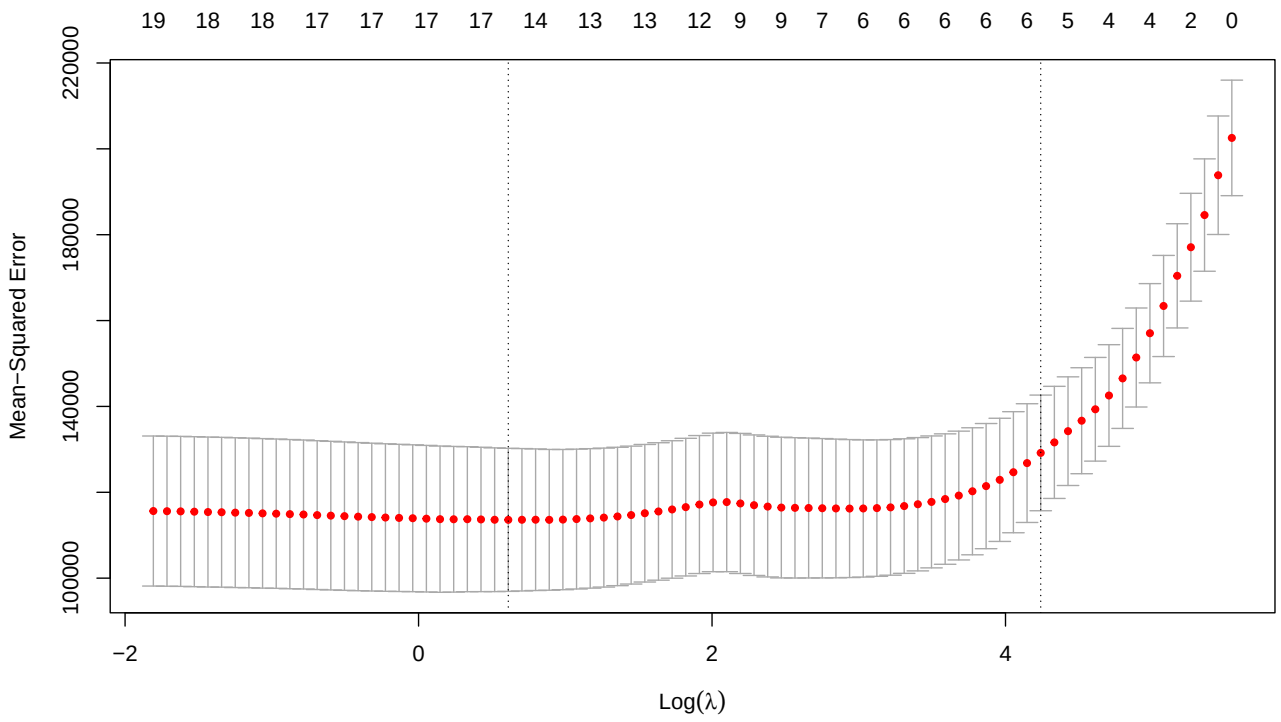
List all lambdas and corresponding CV test deviance (2LL)

```
round(cbind("Lambda" = cv.10Fold.lasso$lambda,
            "10 Fold MSE" = cv.10Fold.lasso$cvm),
      digits = 4)[1:60, ]
```

```
       Lambda 10 Fold MSE
 [1,] 255.2821    202546.7
 [2,] 232.6035    193851.4
 [3,] 211.9397    184575.3
 [4,] 193.1115    177082.0
 [5,] 175.9560    170421.3
 [6,] 160.3246    163395.7
 [7,] 146.0818    157050.7
 [8,] 133.1043    151398.1
 [9,] 121.2797    146527.1
[10,] 110.5055    142542.5
[11,] 100.6885    139334.9
[12,]  91.7436    136669.2
[13,]  83.5934    134237.2
[14,]  76.1672    131622.7
[15,]  69.4007    129171.3
[16,]  63.2353    126797.2
[17,]  57.6177    124674.1
[18,]  52.4991    122897.6
[19,]  47.8352    121439.5
[20,]  43.5857    120233.3
[21,]  39.7136    119237.6
[22,]  36.1856    118422.7
[23,]  32.9710    117758.0
[24,]  30.0419    117215.0
[25,]  27.3731    116788.8
[26,]  24.9413    116481.7
[27,]  22.7256    116314.8
[28,]  20.7067    116229.1
[29,]  18.8672    116206.1
[30,]  17.1911    116232.5
[31,]  15.6639    116281.8
[32,]  14.2723    116336.6
[33,]  13.0044    116375.1
[34,]  11.8491    116442.3
[35,]  10.7965    116668.8
[36,]   9.8374    116994.1
[37,]   8.9634    117400.7
[38,]   8.1672    117720.2
[39,]   7.4416    117640.8
[40,]   6.7805    117145.3
[41,]   6.1782    116540.5
```

```
[42,]   5.6293    115999.3
[43,]   5.1292    115534.9
[44,]   4.6735    115119.2
[45,]   4.2584    114689.6
[46,]   3.8801    114367.9
[47,]   3.5354    114105.7
[48,]   3.2213    113915.2
[49,]   2.9351    113758.7
[50,]   2.6744    113640.1
[51,]   2.4368    113582.0
[52,]   2.2203    113610.7
[53,]   2.0231    113597.8
[54,]   1.8433    113581.6
[55,]   1.6796    113602.7
[56,]   1.5304    113674.8
[57,]   1.3944    113732.1
[58,]   1.2705    113713.6
[59,]   1.1577    113735.3
[60,]   1.0548    113850.6
```

```r
plot(cv.10Fold.lasso) # Plot all lambdas vs. MSEs
```



Let's find the best lambda that minimizes 10FCV deviance

```r
best.lambda.lasso <- cv.10Fold.lasso$lambda.min
log(best.lambda.lasso) # Spot it in the plot
```

```
[1] 0.6115809
```

```r
min.mse.lasso <- min(cv.10Fold.lasso$cvm)

round(cbind("Best Lambda" = best.lambda.lasso,
            "Log(Lambda)" = log(best.lambda.lasso),
            "Best 10FCV MSE" = min.mse.lasso),
      digits = 4)
```

```
     Best Lambda Log(Lambda) Best 10FCV MSE
[1,]      1.8433      0.6116       113581.6
```

Extracting LASSO Coefficients

```r
lasso.0 <- round(coef(lasso.mod, s = 0, # No shrinkage, same as OLS
                      x = x, y = y,
                      exact = T),
                 digits = 4)

lasso.best <- round(coef(lasso.mod, s = best.lambda.lasso, # best Lambda
                         x = x, y = y,
                         exact = T),
                    digits = 4)

lasso.50 <- round(coef(lasso.mod, s = 50), # Some lambda=50
                  digits = 4)

lasso.large <- round(coef(lasso.mod, s = 10 ^ 20), # Null model
                     digits = 4)

lasso.all <- cbind(lasso.0,
                   lasso.best,
                   lasso.50,
                   lasso.large)
colnames(lasso.all) <- c("OLS, Lambda = 0",
                         "Best Lambda = 1.84",
                         "Lambda = 50",
                         "Null")
lasso.all # Display all coefficients
```

```
20 x 4 sparse Matrix of class "dgCMatrix"
            OLS, Lambda = 0 Best Lambda = 1.84 Lambda = 50     Null
(Intercept)        164.2069           142.8776     88.3915 535.9259
AtBat               -1.9947            -1.7934      .        .
Hits                 7.5281             6.1877      1.5868   .
HmRun                4.2484             0.2329      .        .
Runs                -2.3712             .           .        .
RBI                 -1.0073             .           .        .
Walks                6.2563             5.1480      1.8172   .
Years               -3.8760           -10.3928      .        .
CAtBat              -0.1652            -0.0045      .        .
CHits                0.1296             .           .        .
CHmRun              -0.1255             0.5854      .        .
CRuns                1.4418             0.7639      0.1777   .
CRBI                 0.7835             0.3884      0.3646   .
```

```
CWalks             -0.8164           -0.6297      .        .
LeagueN            62.5540           34.2269      .        .
DivisionW        -116.9901         -118.9807   -43.2583    .
PutOuts            0.2822            0.2790      0.1345    .
Assists            0.3694            0.2240      .         .
Errors             -3.3596           -2.4363     .         .
NewLeagueN         -24.9254           .          .         .
```

Let's compare the effects for the best LASSO and Ridge models

```
best.both <- cbind(ridge.best, lasso.best)
colnames(best.both) <- c("Ridge", "LASSO")
best.both # Display all coefficients
```

```
20 x 2 sparse Matrix of class "dgCMatrix"
                Ridge     LASSO
(Intercept)  130.9604  142.8776
AtBat         -1.3235   -1.7934
Hits           4.6658    6.1877
HmRun         -0.2302    0.2329
Runs           0.2324    .
RBI            0.3171    .
Walks          4.6457    5.1480
Years        -10.7666  -10.3928
CAtBat        -0.0330   -0.0045
CHits          0.1779    .
CHmRun         0.7155    0.5854
CRuns          0.5443    0.7639
CRBI           0.3393    0.3884
CWalks        -0.5067   -0.6297
LeagueN       59.9218   34.2269
DivisionW   -123.7976 -118.9807
PutOuts        0.2761    0.2790
Assists        0.2534    0.2240
Errors        -3.8362   -2.4363
NewLeagueN   -26.3324    .
```

Notice again that the OLS and best LASSO models are just about the same. This is not surprising because the best Ridge model was also very close to the OLS model, suggesting that there are no major dimensionality issues, so naturally, the best LASSO model will be close to the OLS model.

Predicting with LASSO

As with Ridge regression, we can use the `predict()` function to obtain LASSO regression coefficients

```
lasso.coef <- predict(lasso.mod,
                      s = best.lambda.lasso,
                      type = "coefficients")
lasso.coef # Notice that the dropped predictors show blanks
```

```
20 x 1 sparse Matrix of class "dgCMatrix"
                        s1
(Intercept)  142.877615618
```

```
AtBat         -1.793369131
Hits           6.187727978
HmRun          0.232913251
Runs           .
RBI            .
Walks          5.147970767
Years        -10.392782086
CAtBat        -0.004469497
CHits          .
CHmRun         0.585358718
CRuns          0.763882254
CRBI           0.388422191
CWalks        -0.629678348
LeagueN       34.226933787
DivisionW   -118.980715736
PutOuts        0.279042882
Assists        0.223985943
Errors        -2.436300484
NewLeagueN     .
```

To make actual predictions and to run LASSO Logistic models, the processes are identical to the ones for Ridge regression above.